Assignment 4: Load Balancing & Scaling on Local Machine

Phạm Trần Gia Hưng – 2331200153

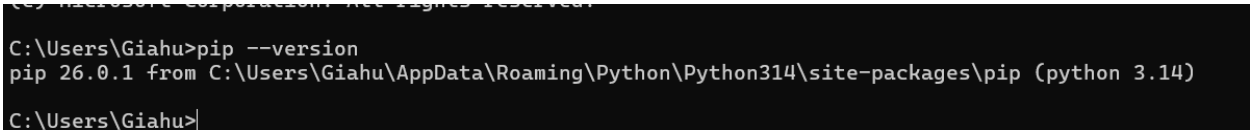
1. Task Completion (70 points total)

Step 1: Go to Python's official website

- Download Python 3.x https://www.python.org/downloads/windows/ • IMPORTANT During installation:
  - Check "Add Python to PATH"

Step 2: Verify installation

- Open Command Prompt (cmd) **or** PowerShell ○ Run the command: python –version ○ Run the command: pip --version ○ Install pip if missing Step 3: Create project folder



- Open command prompt and run this command to create folder ○ mkdir load-balancing-lab
- Navigate to the folder which you have created using the command below:
  - cd load-balancing-lab

Step 4: Create server file

- Open Notepad (or VS Code if installed)
- Save as: server.py

```
from http.server import BaseHTTPRequestHandler, HTTPServer import
sys
PORT = int(sys.argv[1])
INSTANCE = sys.argv[2]

class Handler(BaseHTTPRequestHandler):    def
do_GET(self):
      self.send_response(200)
self.end_headers()
      message = f"Hello from instance {INSTANCE}"
self.wfile.write(message.encode())

server = HTTPServer(("localhost", PORT), Handler) print(f"Running
instance {INSTANCE} on port {PORT}") server.serve_forever()
```

server.py > ...

```python
from http.server import BaseHTTPRequestHandler, HTTPServer
import sys

PORT = int(sys.argv[1])
INSTANCE = sys.argv[2]
class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.end_headers()
        message = f"Hello from instance {INSTANCE}"
        self.wfile.write(message.encode())
server = HTTPServer(("localhost", PORT), Handler)
print(f"Running instance {INSTANCE} on port {PORT}")
server.serve_forever()
```

Step 5: Run multiple servers, and simulate scaling.

- Open 3 Command Prompt individually in your windows machine from different port
  - python server.py 8001 1

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\CSE_programming\CSN305_CloudComputing\CSN305_2331200153_PhamTranGiaHung_lab4> python server.py 8001 1
Running instance 1 on port 8001
```

○ python server.py 8002 2

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

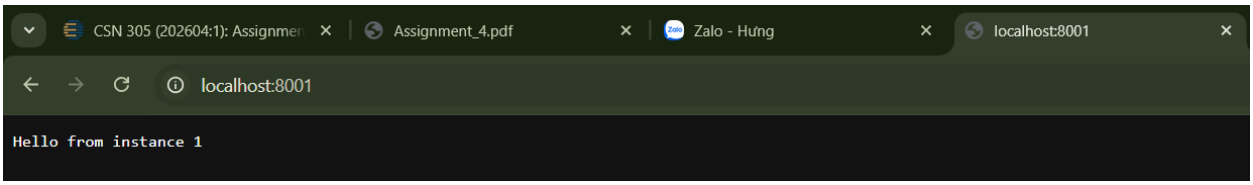
PS C:\CSE_programming\CSN305_CloudComputing\CSN305_2331200153_PhamTranGiaHung_lab4> python server.py 8002 2
Running instance 2 on port 8002
```

○ python server.py 8003 3

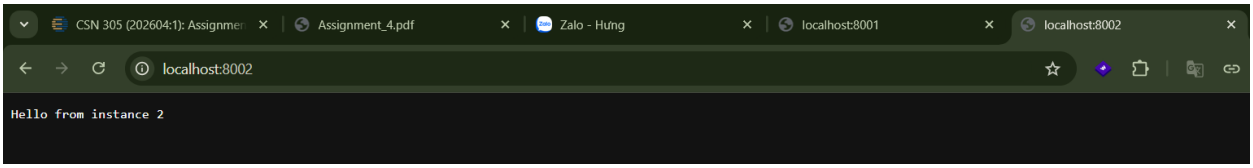
```
PS C:\CSE_programming\CSN305_CloudComputing\CSN305_2331200153_PhamTranGiaHung_lab4> python server.py 8003 3
Running instance 3 on port 8003
```

Step 6: Open browser and Test in browser

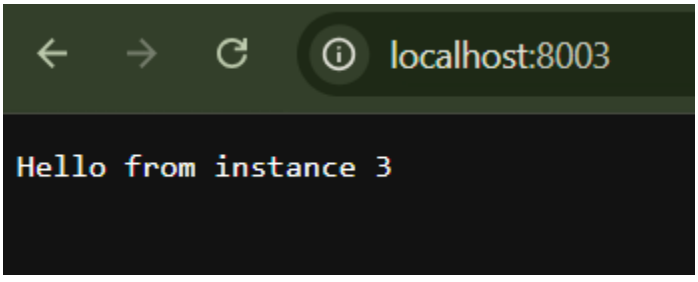
- http://localhost:8001



- http://localhost:8002



- http://localhost:8003



Each shows different instance

Step 7: open another command prompt and Install requests library

- pip install requests

```
PS C:\CSE_programming\CSN305_CloudComputing\CSN305_2331200153_PhamTranGiaHung_lab4> pip install requests
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: requests in C:\Users\Giahu\AppData\Roaming\Python\Python314\site-packages (2.33.1)
Requirement already satisfied: charset_normalizer<4,>=2 in C:\Users\Giahu\AppData\Roaming\Python\Python314\site-packages
 (from requests) (3.4.7)
Requirement already satisfied: idna<4,>=2.5 in C:\Users\Giahu\AppData\Roaming\Python\Python314\site-packages (from reque
sts) (3.11)
Requirement already satisfied: urllib3<3,>=1.26 in C:\Users\Giahu\AppData\Roaming\Python\Python314\site-packages (from r
equests) (2.6.3)
Requirement already satisfied: certifi>=2023.5.7 in C:\Users\Giahu\AppData\Roaming\Python\Python314\site-packages (from
requests) (2026.2.25)

[notice] A new release of pip is available: 26.0.1 -> 26.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\CSE_programming\CSN305_CloudComputing\CSN305_2331200153_PhamTranGiaHung_lab4>
```

Step 8: Create load balancer

• Create a load_balancer.py file.

```
import itertools
from http.server import BaseHTTPRequestHandler, HTTPServer import
requests

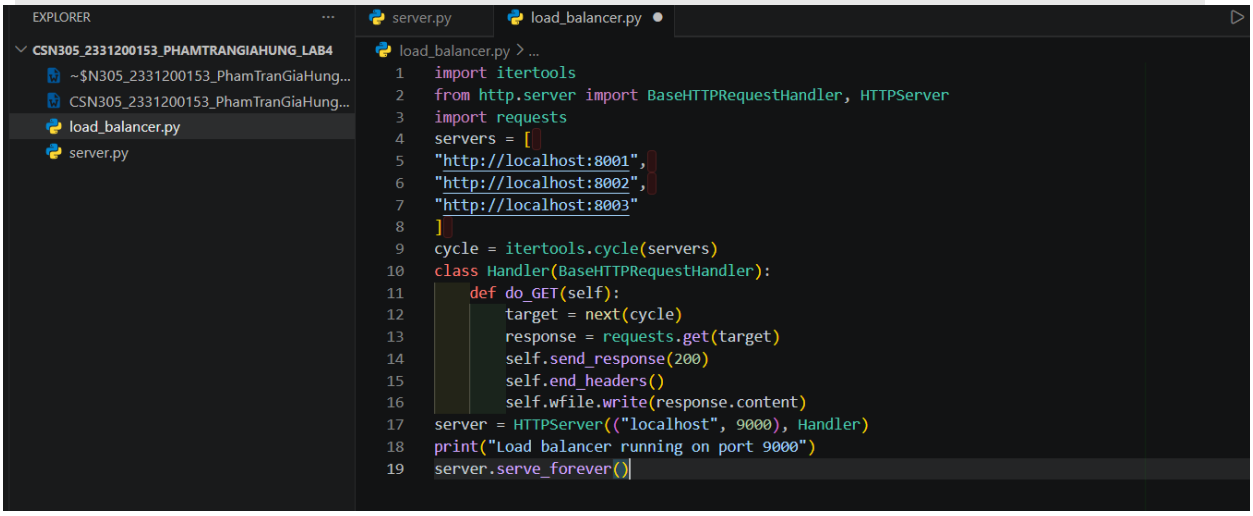
servers = [
   "http://localhost:8001",
   "http://localhost:8002",
   "http://localhost:8003"
]

cycle = itertools.cycle(servers)

class Handler(BaseHTTPRequestHandler):    def
do_GET(self):
      target = next(cycle)
      response = requests.get(target)

      self.send_response(200)
self.end_headers()
      self.wfile.write(response.content)

server = HTTPServer(("localhost", 9000), Handler) print("Load
balancer running on port 9000") server.serve_forever()
```



Step 9: Open another command prompt and Run load balancer

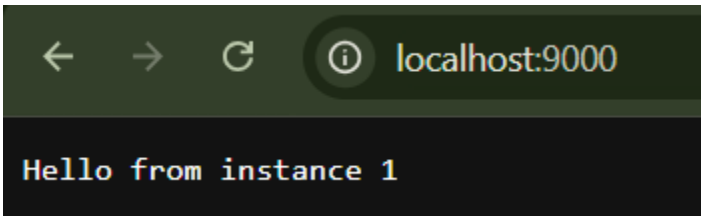
- python load_balancer.py

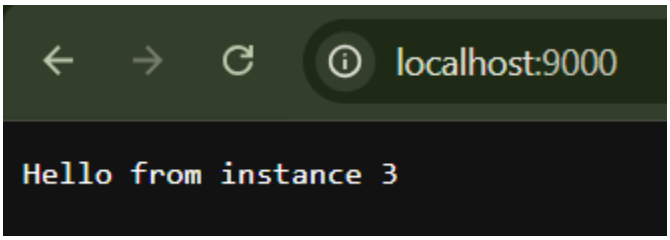
```
PS C:\CSE_programming\CSN305_CloudComputing\CSN305_2331200153_PhamTranGiaHung_lab4> python load_balancer.py
Load balancer running on port 9000
```

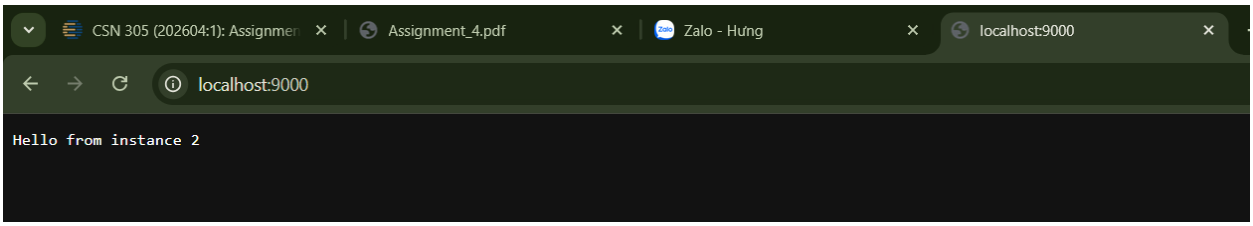
Step 10: Demonstrate load balancing

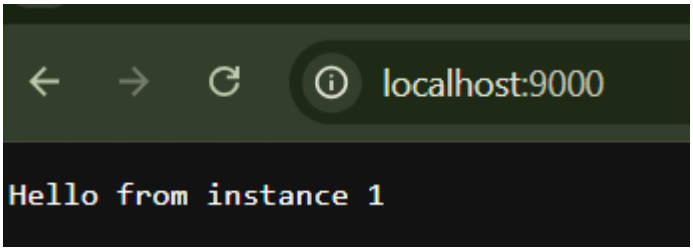
- Open browser and test the following URL http://localhost:9000

- Refresh multiple times. You'll see:
  - Hello from instance 1
  - Hello from instance 2
  - Hello from instance 3









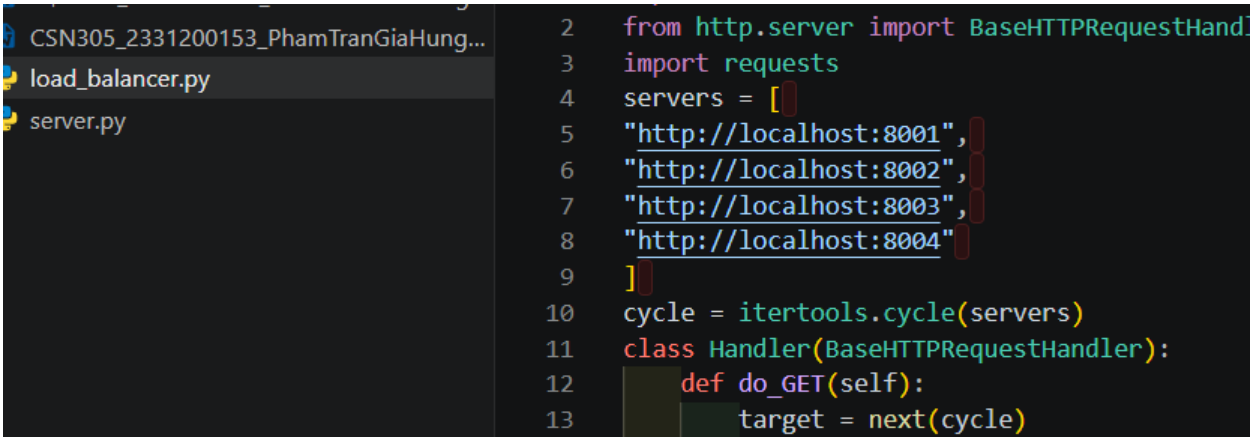
- This proves load balancing works on local machine. This is round-robin load balancing. Requests are distributed evenly

Step 11: Demonstrate Scaling

- More servers = more capacity. So add one more server.
- Open new command prompt and run *python server.py 8004 4*

```
PS C:\CSE_programming\CSN305_CloudComputing\CSN305_2331200153_PhamTranGiaHung_lab4> python server.py 8004 4
Running instance 4 on port 8004
```

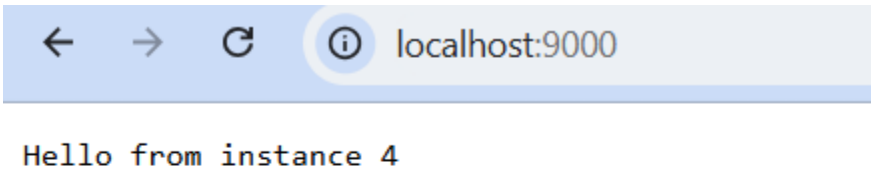
- Update the file load_balancer.py by:
    - servers = [
      "http://localhost:8001",
      "http://localhost:8002",
      "http://localhost:8003",
      "http://localhost:8004"
      ]



- Now restart the load balancer again by *python load_balancer.py*

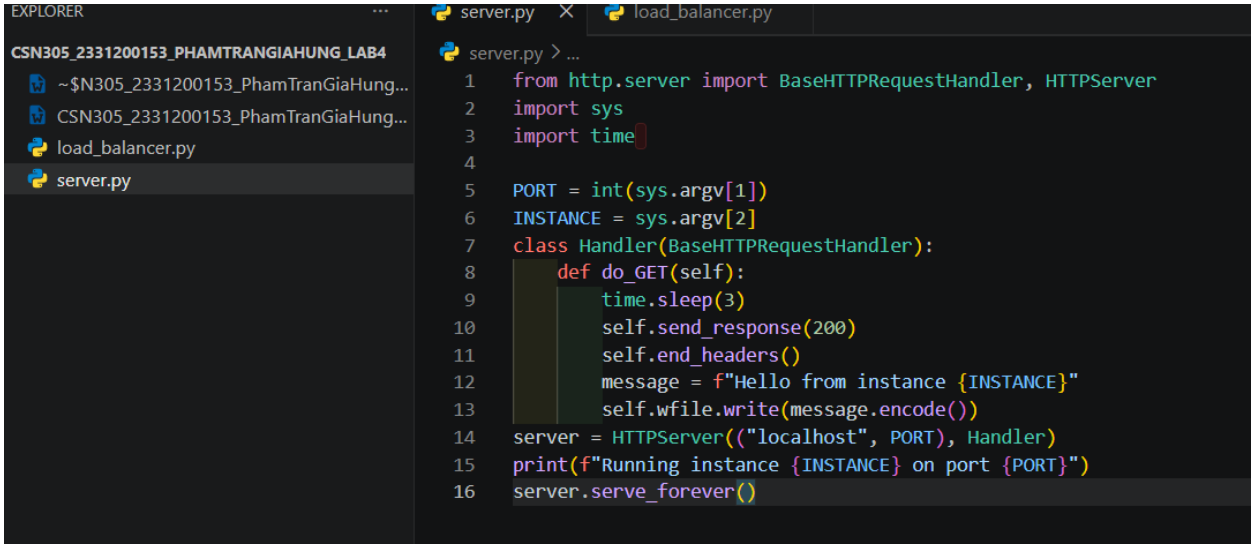
```
PS C:\CSE_programming\CSN305_CloudComputing\CSN305_2331200153_PhamTranGiaHung_lab4> python load_balancer.py
Load balancer running on port 9000
```

- Now 4 servers handle traffic



Step 12: Demonstrate Slow Server Problem.

- Modify server.py. by
  - import time time.sleep(3)



- Restart instance 2 (Test only for instance 2). Sometimes response is slow. Explain uneven performance

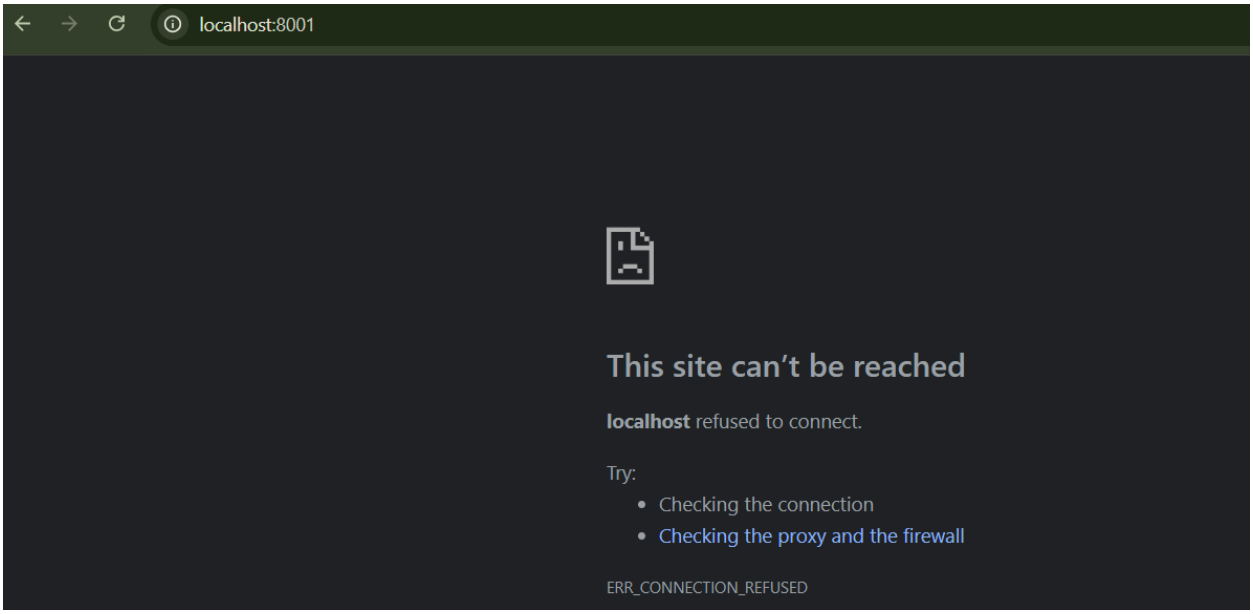
- The round-robin load balancer distributes requests blindly it dont have awareness of how long each server takes to respond. It simply cycles through the list in order. So when instance 2 is slow, the load balancer still sends it the same share of traffic, it assumes all servers are equally capable and equally fast at all times

Step 13: Demonstrate Failure Handling

- Stop any one server by
  - Ctrl + C

```
PS C:\CSE_programming\CSN305_CloudComputing\CSN305_2331200153_PhamTranGiaHung_lab4> python server.py 8001 1
Running instance 1 on port 8001
127.0.0.1 - - [05/May/2026 08:02:24] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [05/May/2026 08:02:27] "GET /favicon.ico HTTP/1.1" 200 -
127.0.0.1 - - [05/May/2026 08:06:25] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [05/May/2026 08:07:05] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [05/May/2026 08:07:44] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [05/May/2026 08:13:41] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [05/May/2026 08:14:01] "GET / HTTP/1.1" 200 -
Traceback (most recent call last):
  File "C:\CSE_programming\CSN305_CloudComputing\CSN305_2331200153_PhamTranGiaHung_lab4\server.py", line 16, in <module>
    server.serve_forever()
    ~~~~~~~~~~~~~~~~~~~~^^
  File "C:\Python314\Lib\socketserver.py", line 235, in serve_forever
    ready = selector.select(poll_interval)
  File "C:\Python314\Lib\selectors.py", line 314, in select
    r, w, _ = self._select(self._readers, self._writers, [], timeout)
              ~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Python314\Lib\selectors.py", line 305, in _select
    r, w, x = select.select(r, w, w, timeout)
              ~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^
KeyboardInterrupt
PS C:\CSE_programming\CSN305_CloudComputing\CSN305_2331200153_PhamTranGiaHung_lab4>
```

- Now refresh browser. You will see it may crash or hang.



- Explain real systems must handle failures

- Health checks: periodically pinging each backend; if it fails to respond, it is removed from the pool automatically

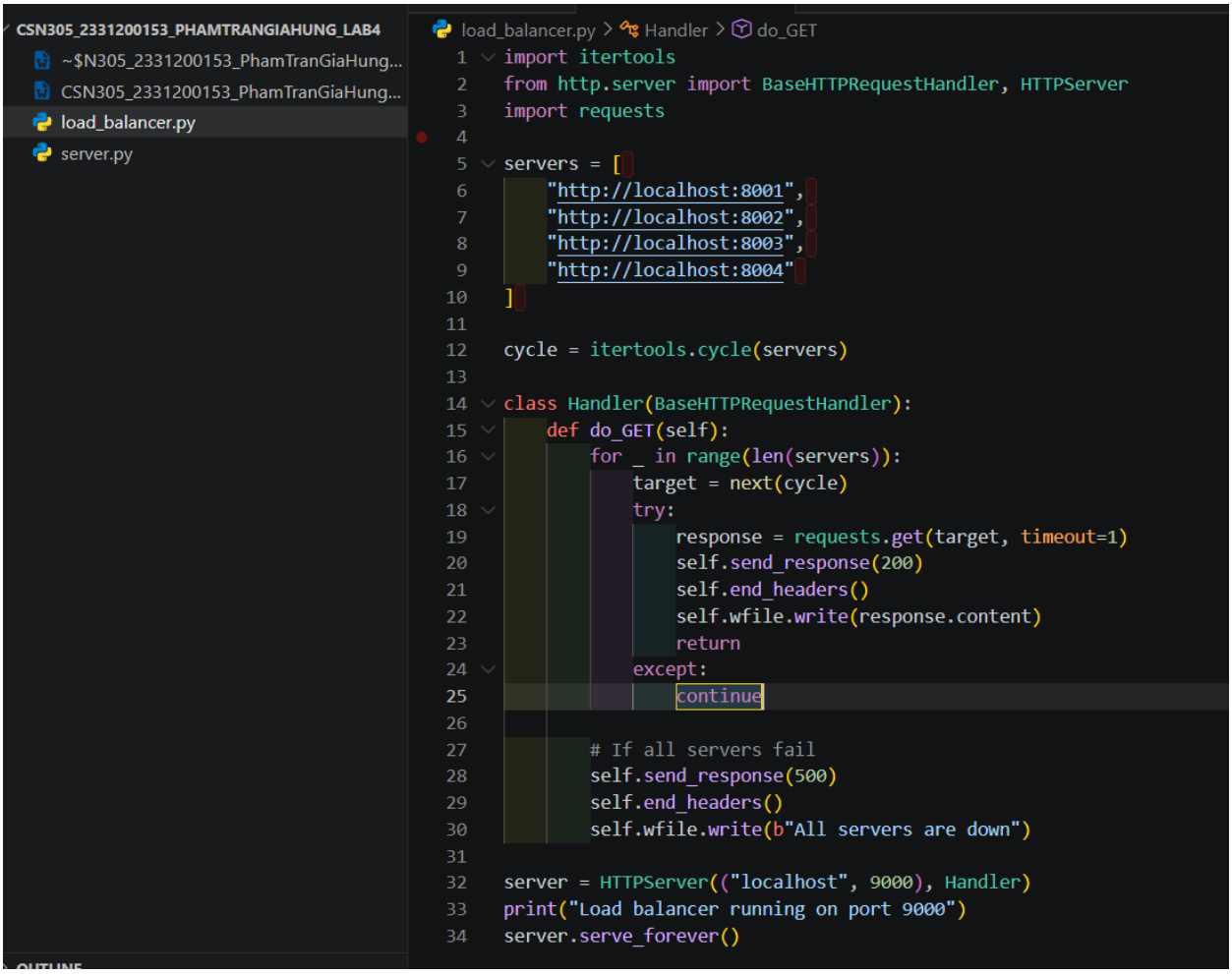
- Automatic failover: traffic is instantly rerouted to the remaining healthy servers without any visible error

-Retry logic: like the try/except, which retries the next available server before giving up

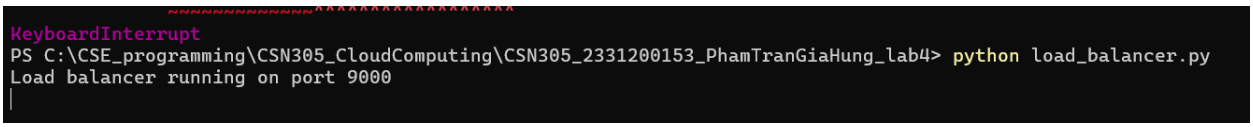
Step 14: Improve Load Balancer (basic fault tolerance)

- By taking this step dead servers are skipped.
- Update the load balancer by modifying do_GET method in load_balancer.py class Handler(BaseHTTPRequestHandler): def do_GET(self): for _ in range(len(servers)):

```
target = next(cycle)
try:
response = requests.get(target, timeout=1)
self.send_response(200) self.end_headers()
self.wfile.write(response.content)
return except:
continue # If all
servers fail
self.send_response(500)
self.end_headers()
self.wfile.write(b"All servers are down")
```



- Stop and restart python load_balancer.py
    - You will observe if one server is down → system still works



    - If all servers are down → error message shown

This is a real-world concept used in systems such as AWS load balancer, NGINX etc webserver.